

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

1. Část projektu IDS

zadání IUS 37. Rezervace letenek

Petr Kaška (xkaska01)
Martin Hemza (xhemza05)

1 Zadání:

Vaším úkolem je navrhnout webovou aplikaci pro rezervaci letenek. Systém musí uživateli umožnit specifikovat požadavek pomocí místa odletu a příletu, data, času, třídy, letecké společnosti apod. Jedno letadlo může létat na více letech, stejně tak mezi dvěma destinacemi může létat více letadel různých společností. Zákazník si rezervuje letenku, která může být na několik letů, i různých společností (např. letenka Praha → San Francisco s lety Praha → Londýn ČSA, Londýn → San Francisco British Airways). Systém musí umožnit klientovi tisk letového itineráře, který obsahuje informace o dobách příletů a odletů na jednotlivých letištích. Každý typ letadla má různý počet sedadel a jejich rozvržení do tříd. Nemodelujte jednotlivá sedadla. Rezervací může klient zamluvit i více míst v jednom letu. Technická mezipřistání modelovat nemusíte. Systém musí evidovat, která společnost kdy, odkud a kam létá. Cena sedadla v dané třídě může být u každé společnosti jiná, cena letenky je dána součtem cen všech rezervovaných sedadel na všech letech. Pokud klient rezervovanou letenku včas nezaplatí, rezervace se z databáze smaže.

2 Popis ERD:

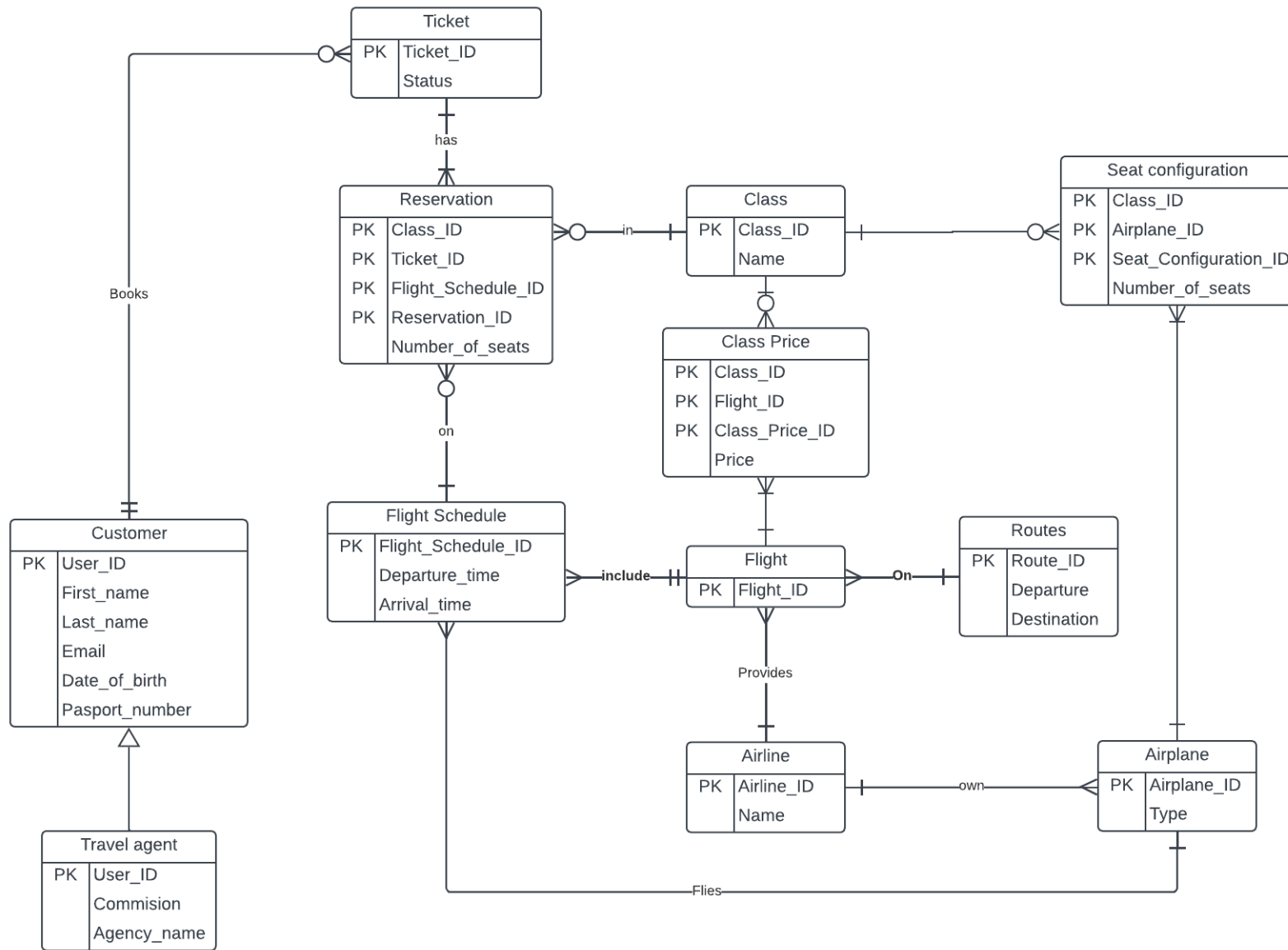
Tento systém rezervace letenek je rozdělen do devíti entitních množin, jedné generalizace/specializace a třemi vazebními vztahy. Existují dva druhy uživatelů: zákazník a cestovní agent. Pro cestovního agenta jsou k dispozici některé atributy, které zákazník nemá, a proto jsme použili generalizaci/specializaci pro cestovního agenta a zákazníka.

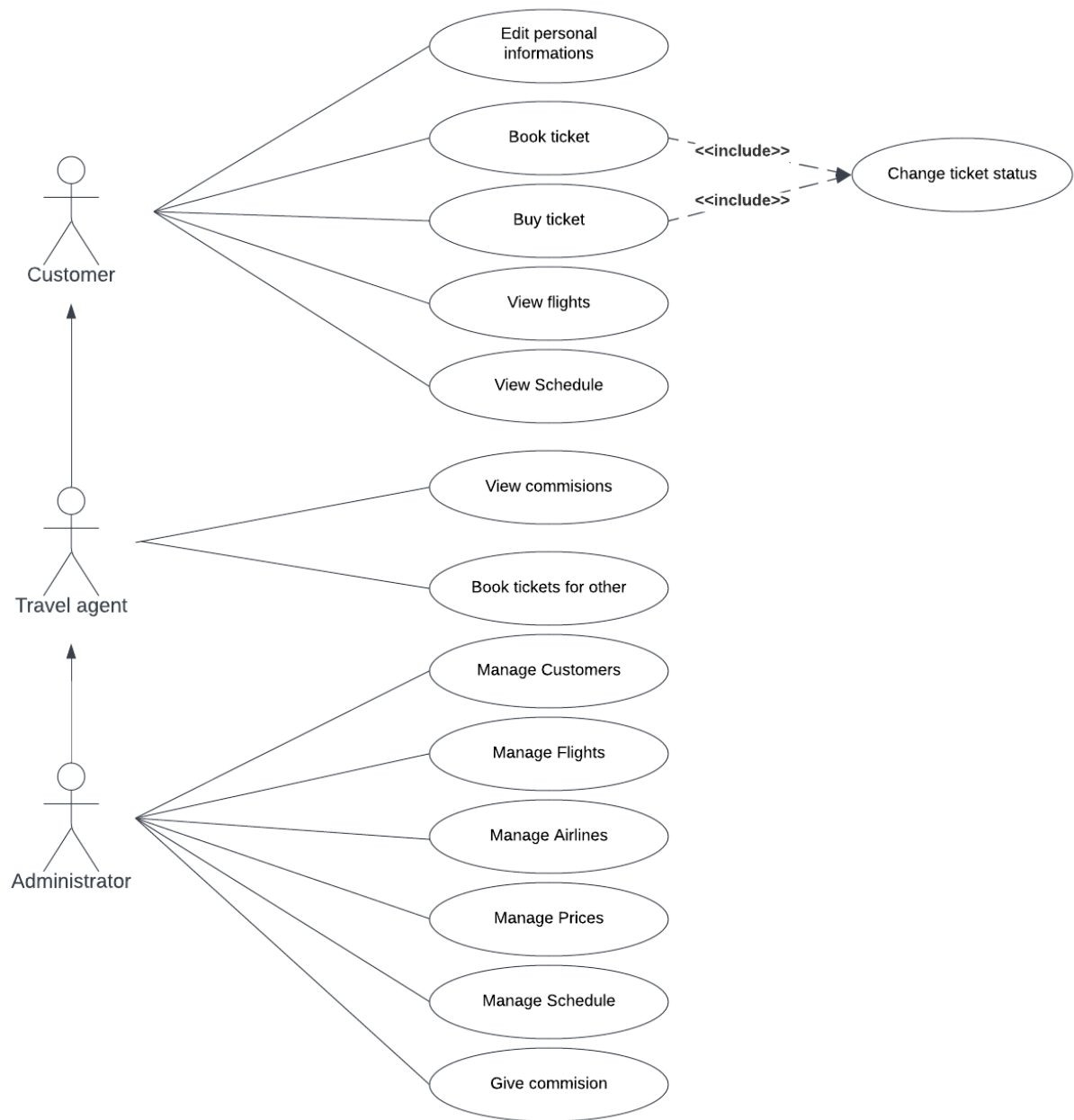
Systém obsahuje silnou entitu "lístek", kde každý zákazník může koupit více lístků, ale každý konkrétní lístek je vždy pro jednoho zákazníka. Toto jsme modelovali pomocí vztahu 1:N. Dále je k dispozici silná entitní množina "třída", vazební množina "rozložení sedadel" a silná entitní množina "letadlo", kde instance právě jedné třídy a právě jednoho letadla musí vždy mít stejný počet sedadel. Dále jsme navrhli silnou entitní množinu "aerolinka", protože aerolinky obvykle mají více letadel a víc letů, takže jsme použili vztahy 1:N. Další silnou entitní množinou jsou "let" a "trať". Trasy se obvykle často nemění a létá po nich více letadel různých aerolinií, proto jsme to namodelovali tímto způsobem. Vazební množina "cena třídy" udává cenu jednotlivých lístků v různých třídách jednoho letu.

Následně jsme modelovali Množinu Letový plán, díky které jsme schopni zaznamenat časy přistání a odletů jednotlivých letů.

3 Popis Use Case:

Systém se skládá ze 3 aktérů zákazníka, cestovního agenta a administrátora. U zákazníka jsme použili include. Abychom zdůraznily změnu stav letenky na koupená/rezervovaná při interakci s nimi.





4 SQL skript pro vytvoření pokročilých objektů schématu databáze

4.1 Triggery

4.1.1 dolar_convertor

Trigger, který slouží k převadení ceny jednotlivých tríd z korun na dolary, spouští se při vložení nové tridy

4.1.2 date_of_birth_check

Trigger, který zkontroluje zda jsou data narození jednotlivých klientů správná (dny v jednotlivých měsících, přestupný rok, ...). Spouští se při vložení zákaznických dat narození

4.1.3 update_seats_available

Trigger slouží k aktualizaci počtu dostupných sedadel v tabulce **SEAT_CONFIGURATION** na základě počtu rezervovaných sedadel v tabulce **RESERVATION**. Pokud se jedná o operaci vkládání nebo aktualizace, počet rezervovaných sedadel se získá z řádku **NEW**. Pokud jde o operaci mazání, počet rezervovaných sedadel se získá jako opačná hodnota počtu sedadel v řádku **NEW**. Poté je provedena aktualizace počtu sedadel v tabulce **SEAT_CONFIGURATION**. Řádek s odpovídajícím **AIRPLANE_ID** je vyhledán pomocí poddotazu, který získává hodnotu **AIRPLANE_ID** z tabulky **FLIGHT_SCHEDULE** na základě hodnoty **FLIGHT_SCHEDULE** z řádku **NEW**. Aktualizace je provedena pouze pro řádky, kde **CLASS_ID** odpovídá hodnotě **CLASS_ID** z řádku **NEW**.

4.2 Procedury

4.2.1 show_customers_on_route

procedura, která vypíše jména klientů, kteří mají lístek na příslušné trase `show_customers_on_route` (odletová destinace, příletová destinace)

4.2.2 print_customer_names

procedura, která vypíše všechna jména zákazníků, kteří jsou momentálně v databázi

4.3 Explain plan

`Explain plan` nám umožňuje zjistit postupnost a počet kroků, které se použijí při přístupu k údajům v systému relačních databází. V našem skriptu jsme zjistili posloupnost kroků pro dotaz **Kolik průměrně utratil zákazník s id 2 za letenky**. Následně jsme vykonali optimalizaci pomocí vytvoření indexu pro sloupec **ID_uživatele** z tabulky **Ticket** a index pro sloupec, který v tabulce **Class_price** uchovává id jednotlivých tříd **id_třídy**. Díky přístupu přímo k indexům se vyhledávání urychlilo, protože nebylo v daných případech nutné přistupovat k celé tabulce. Úspěšnou optimalizaci jsme ověřili opětovným použitím příkazu po optimalizaci na stejný dotaz

4.4 Přístupová práva

Přístupová práva pro druhého člena týmu jsme vyřešili pomocí **GRANT ALL** pro všechny tabulky definované v naší databázi, procedury a materializovaný pohled. Tím pádem je druhý uživatel také administrátor, protože má plný přístup

4.5 Materializovaný pohled

Vytvořili jsme materializovaný pohled s názvem **Agent_view**, který obsahuje sloupce **first_name**, **last_name** a **commision** ze dvou tabulek **Travel_Agent** a **Customer**. Pohled bude automaticky obnovován po každé provedené transakci. Následující Select vypisuje obsah materializovaného pohledu. Poté provedeme UPDATE, který aktualizuje hodnotu **commision** agenta s id 1 v tabulce **Travel_Agent**.

4.6 Transakce

4.6.1 tran. 1

tato transakce vloží nového zákazníka "Johna Doe" a poté vytvoří jeho lístek se statusem "un-booked", vytvoří mu 1 rezervaci do ekonomické třídy a ticket status nastaví na booked. Atomicnost - BEGIN(zajistit provedení kódu jako celku), COMMIT(dokončí transakci) a ROLLBACK(v případě nedokončení transakce vrátí zpět změny a udrží konzistenci databáze)

4.6.2 tran. 2

Vybereme řádek se zákazníkem, který má ID 1 a ten uzamkneme (Pote změníme jméno zákazníka na Jirka a vypíšeme zákazníka do Dbms Outputu a commitneme změny). FOR UPDATE (uzamkne vybraný řádek, aby s ním mohl pracovat pouze 1 uživatel)

4	Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
5							
6	0	SELECT STATEMENT		1	197	11 (19)	00:00:01
7	1	SORT ORDER BY		1	197	11 (19)	00:00:01
8	2	HASH GROUP BY		1	197	11 (19)	00:00:01
9	* 3	HASH JOIN		1	197	10 (10)	00:00:01
10	4	NESTED LOOPS		1	158	7 (15)	00:00:01
11	5	NESTED LOOPS		4	158	7 (15)	00:00:01
12	6	NESTED LOOPS		4	528	6 (17)	00:00:01
13	7	TABLE ACCESS BY INDEX ROWID	Customer	1	67	2 (0)	00:00:01
14	* 8	INDEX UNIQUE SCAN	SYS_C003324744	1		1 (0)	00:00:01
15	9	VIEW	VM_GBF_16	4	260	4 (25)	00:00:01
16	10	SORT GROUP BY		4	156	4 (25)	00:00:01
17	11	TABLE ACCESS FULL	Reservation	4	156	3 (0)	00:00:01
18	* 12	INDEX UNIQUE SCAN	SYS_C003324780	1		0 (0)	00:00:01
19	* 13	TABLE ACCESS BY INDEX ROWID	Ticket	1	26	1 (0)	00:00:01
20	14	TABLE ACCESS FULL	Class_Price	4	156	3 (0)	00:00:01

Obrázek 1: Před optimalizací

4	Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
5							
6	0	SELECT STATEMENT		1	171	8 (13)	00:00:01
7	1	SORT ORDER BY		1	171	8 (13)	00:00:01
8	2	HASH GROUP BY		1	171	8 (13)	00:00:01
9	3	NESTED LOOPS		1	171	7 (0)	00:00:01
10	4	NESTED LOOPS		1	171	7 (0)	00:00:01
11	* 5	HASH JOIN		1	132	6 (0)	00:00:01
12	6	NESTED LOOPS		1	93	3 (0)	00:00:01
13	7	TABLE ACCESS BY INDEX ROWID	Customer	1	67	2 (0)	00:00:01
14	* 8	INDEX UNIQUE SCAN	SYS_C003324744	1		1 (0)	00:00:01
15	9	TABLE ACCESS BY INDEX ROWID BATCHED	Ticket	1	26	1 (0)	00:00:01
16	* 10	INDEX RANGE SCAN	IDX_TICKET_USER_ID	1		0 (0)	00:00:01
17	11	TABLE ACCESS FULL	Reservation	4	156	3 (0)	00:00:01
18	* 12	INDEX RANGE SCAN	IDX_CLASS_PRICE_CLASS_ID	1		0 (0)	00:00:01
19	* 13	TABLE ACCESS BY INDEX ROWID	Class_Price	1	39	1 (0)	00:00:01

Obrázek 2: Po optimalizaci